

Databases and ORMs

FastAPI Masterclass

Our Current State

- We have a FastAPI application with multiple endpoints that return room data.
- Different clients (a web application, an iOS app, etc) can make requests to the endpoints to fetch data.
- The data is *hardcoded*. We defined our rooms inline as plain Python objects, so they are lost/cleared from memory when the server shuts down.

Data in the Real World

- Data in the real world lives in a database, a more permanent solution for data storage.
- The database is optimized for extracting records.
- Backend applications query the database for data.
- If the backend restarts, the data is not lost.
- **Database = Data. Backend = Business Logic.**

Relational Databases

- A **relational database** organizes data in tables.
- A table is a 2-dimensional grid with columns (the fields) and rows (the individual records).
- Tables establish associations/relations to other tables by linking IDs.
- Tables are comparable to spreadsheets.

Sample Table: rooms

id	name	price_per_night	bedrooms	bathrooms
1	House	350	5	3.5
2	Villa	785	3	2
3	Studio	200	1	1

SQLite

- SQLite is a small, lightweight relational database that easily integrates with Python.
- SQLite saves the database contents to a single file.
- For a real-world application, we'd go with a more production-ready solution like Postgres.
- Postgres handles high concurrency well (multiple read/writes at the same time).

The Challenge

- Our FastAPI backend is written in Python.
- A database like Postgresql or SQLite uses Structured Query Language (SQL).
- **SQL** is the language that interacts with a relational database -- creating tables, inserting records, and running queries.

Object Relational Mapper [ORM]

- An **object-relational mapper (ORM)** is a library that acts as a translation layer between a programming language and SQL.
- The ORM translates Python code into equivalent SQL statements.
- Vice versa, the ORM converts database records into Python object representations.

FastAPI and ORMs

- FastAPI is a micro-framework with less opinions than other backend solutions.
- FastAPI does not force you to adopt a specific ORM.
- ORM options in the Python ecosystem include SQLAlchemy, SQLModel, Tortoise, and more.
- We'll choose **SQLModel** for the simple reason that it is built by the same development team as FastAPI.

SQLModel vs Pydantic

- SQLModel is built on top of *both* the Pydantic data validation library and the SQLAlchemy ORM.
- Pydantic validates that payloads fit the schema of a model (including type conversion).
- SQLModel merges Pydantic's model validation with SQLAlchemy's database communication, and then adds a layer of functionality on top.